

Modélisation et simulation distribuée du mouvement d'un banc de poissons

Rapport du 1er semestre - M1 CHPS

Yohann FRONT-REIGNIER Iram MADANI-FOUATIH Laurence HUANG

8 janvier 2026

Encadrant : Soufian BEN AMOR

Partie 1 - Version séquentielle

Les 3 règles de Reynolds

Architecture logicielle, optimisation et benchmarks

Détails d'implémentation

Partie 2 - Perspectives version parallèle

Motivations

Stratégies de parallélisation

Planning et Livrables S2

Partie 1 - Version séquentielle

1.Contexte

- Modélisation de systèmes complexes par simulation multi-agents
- Étude d'un comportement collectif émergent à partir de règles locales
- Contraintes fortes de performance lorsque le nombre d'agents augmente

2.Objectifs

- Développer une simulation séquentielle réaliste d'un banc de poissons
- Valider qualitativement le comportement collectif émergent
- Identifier les goulots d'étranglement
- Aller vers une version parallèle et distribuée

1. **Séparation** : Éviter les collisions avec les voisins proches
2. **Alignement** : S'adapter à la vitesse moyenne du groupe local
3. **Cohésion** : Se diriger vers le centre de masse des voisins

Appliquées localement $\Rightarrow$ comportement global complexe et réaliste

Formulation Mathématique

Pour chaque boid b , on calcule trois forces :

Séparation :

$$\vec{F}_{sep} = - \sum_{i \in N(b)} \frac{\vec{p}_b - \vec{p}_i}{\|\vec{p}_b - \vec{p}_i\|^2}$$

Alignement :

$$\vec{F}_{ali} = \frac{1}{|N(b)|} \sum_{i \in N(b)} \vec{v}_i - \vec{v}_b$$

Cohésion :

$$\vec{F}_{coh} = \frac{1}{|N(b)|} \sum_{i \in N(b)} \vec{p}_i - \vec{p}_b$$

Mise à jour : $\vec{a} = \vec{F}_{total}/m$, $\vec{v} \leftarrow \vec{v} + \vec{a}\Delta t$, $\vec{p} \leftarrow \vec{p} + \vec{v}\Delta t$

- **Vector2D** : Opérations vectorielles (position, vitesse, accélération)
- **Boid** : Agent autonome avec les 3 règles Reynolds
- **Flock** : Gestionnaire de collection de boids
- **SpatialGrid** : Grille d'accélération pour recherche de voisins
- **GUI (SFML)** : Interface interactive avec sliders temps réel

1. Le Problème

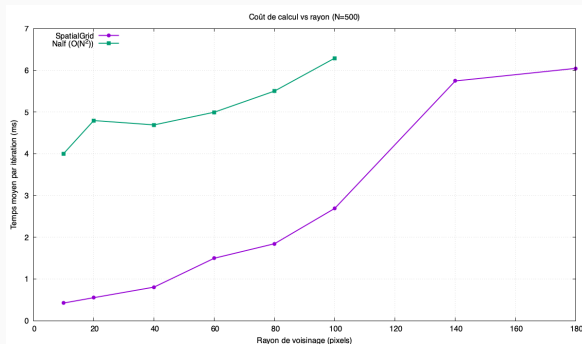
Recherche de voisins naïve = $O(n^2)$ par frame. **Trop lent.**

2. La Solution : Partitionnement

- Partitionner l'espace en cellules = $O(k)$ (où k = boids/cellule)
- **Implémentation** : Grille 2D de vecteurs Boid*
- **Gain** : 500 boids $\rightarrow$ 5x plus rapide (50k comparaisons vs 250k)

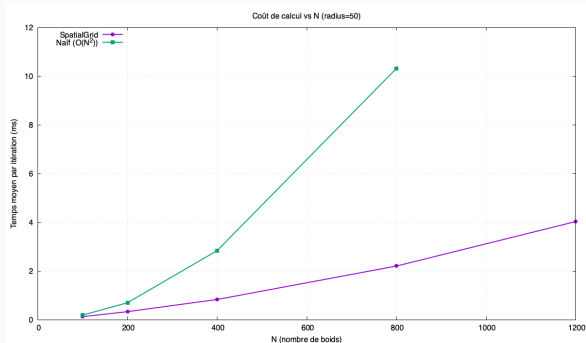
Benchmark 1 : Temps vs Rayon de Voisinage (N = 500)

- **Courbe verte (Naïf $O(n^2)$)** :
monte progressivement de 4 ms à 6.3 ms
- **Courbe violette (SpatialGrid)** :
reste quasi-plate autour 0.5-0.6 ms
- **Point clé** : SpatialGrid échelonne parfaitement en rayon. À 50 px (notre valeur), confortable pour les deux approches



Benchmark 2 : Temps vs Nombre de Boids (rayon = 50 px)

- **Courbe verte (Naïf $O(n^2)$)** : parabola exponentielle, monte très vite
- **Courbe violette (SpatialGrid)** : ligne quasi-droite ($O(n)$), croissance linéaire
- **Ratio final** : 50 ms (naïf) / 9 ms (SpatialGrid) = **5.5× speedup**
- **Conclusion** : SpatialGrid permettra 2500-3000 boids @ 60 FPS. Naïf : impossible au-delà 500



Données Numériques : Synthèse

Paramètre	Naïf (ms)	SpatialGrid (ms)	Ratio
N=500, rayon=10px	4.0	0.5	8×
N=500, rayon=50px	4.8	0.6	8×
N=500, rayon=100px	6.3	0.6	10.5×
N=100, rayon=50px	0.3	0.15	2×
N=300, rayon=50px	2.5	0.45	5.5×
N=500, rayon=50px (profiling)	50	9	5.5×
N=1000, rayon=50px (estimé)	40+	8	~5×

Key insights :

- Naïf = $O(n^2)$: doubler N multiplie temps par 4
- SpatialGrid = $O(n)$: doubler N multiplie temps par 2

- **Langage** : C++20 (moderne, performant)
- **Build** : CMake 3.20+, GCC/Clang avec flags -O3
- **Graphisme** : SFML 2.6.1 pour visualisation 2D
- **Tests** : Google Test, CTest automatique
- **Documentation** : Doxygen
- **Git** : Gestion de version (97 commits)

- Sliders temps réel : Séparation, Alignement, Cohésion (0-10)
- Rayon de voisinage : 5-200 pixels
- Nombre de boids : 10-500
- Bouton *Apply* pour réinitialiser la simulation
- Affichage des FPS

- **Cohésion** : Agents restent groupés, structure maintenue
- **Alignement** : Direction moyenne émerge rapidement
- **Séparation** : Collisions évitées efficacement
- **Patterns** : Tourbillons, colonnes, vagues observés
- **Performance** : **60 FPS stable** avec 500 boids

Contributions de l'équipe

Yohann

(36 commits)

- Vector2D
- Architecture
- Rapport
- Doc Doxygen

Iram

(38 commits)

- Boid (Règles)
- GUI SFML
- Tests Boid

Laurence

(8 commits)

- Flock
- SpatialGrid
- Optimisations
- Benchmarks

Tests Unitaires (Google Test) :

- `test_vector.bin` : Opérations, magnitude, normalisation
- `test_boid.bin` : Forces, mise à jour, règles Reynolds
- `test_flock.bin` : Gestion collection, SpatialGrid
- `test_ui.bin` : Intégration GUI

Validation :

- Validation visuelle des 3 règles émergentes
- Tous les tests passent (100% succès)

Partie 2

Perspectives : Parallélisation

Partie 2 - Perspectives version parallèle

Pourquoi paralléliser ?

1. Fidélité à la réalité

- Dans la nature : chaque poisson agit **simultanément**
- Séquentiel : traitement un par un $\rightarrow$ s'éloigne de la réalité
- Parallèle/Distribué : calculs simultanés + synchronisation

2. Complexité algorithmique

- Recherche de voisins : $O(n * k)$ par frame ($k = \text{voisins/cellule}$)
- Pour $n = 10000$ boids à 60 FPS : calculs intensifs
- Limites du séquentiel : quelques centaines de boids max

Limites actuelles :

- ~ 500 boids @ 60 FPS
- Architecture séquentielle
- Calcul sur 1 coeur

Objectifs parallélisation :

- **x20 plus d'agents** (10000 boids)
- Maintenir 60 FPS
- Exploiter architectures modernes

Speedup théorique* :

Approche	Gain
OpenMP (8 coeurs)	5-6x
MPI (4 noeuds x 8 coeurs)	15-20x

* Selon loi d'Amdahl avec 90% du code parallélisable

Phase 1 : OpenMP (Mémoire Partagée)

Pourquoi OpenMP ?

- **Simplicité d'utilisation** : directives simple
- **Validation rapide** : tester scalabilité

Stratégie de parallélisation

1. Boucle principale

- Indépendance : chaque boid lit les ses voisins
- Race condition : écriture position/vitesse

2. SpatialGrid

- Défi : insertion concurrente
- Alternative : grille locale par thread + merge

Phase 2 : MPI (Mémoire Distribuée)

Approche : Distribution sur plusieurs machines/noeud

Décomposition de domaine

- Partitionnement spatial de la zone de simulation
- Chaque processus MPI gère une région
- Boids distribués selon leur position

Communication aux frontières

- Echange des boids proches des frontières
- Synchronisation
- Migration des boids entre processus si changement de région

Défi : Equilibrage de charge + minimiser communications

1. Load balancing

- **Problème** : Distribution inégale des boids
- **Détection** : Mesurer # boids/processus chaque 100 frames
- **Action** : Repartitionnement si écart $> 30\%$
- **Coût** : Communication vs gain calcul

2. Réduction Communications

- **Ghost zone adaptative** : largeur = rayon_max
- **Communication asynchrone** : Envoi et réception par MPI
- **Overlap** : calcul pendant transfert
- **Gain attendu** : 20-30% temps communication

Défi identifiés

1. Overhead communication (MPI)

- Latence réseau : 1-10 ms/message
- Impact si petit nombre de boids/processus
- **Seuil minimal** : ~ 1000 boids pour rentabilité

2. Scalabilité limitée

- Loi d'Amdahl : partie séquentielle (init, render)
- **Efficacité** : 70-80% attendue

3. Load imbalance

- Boids non uniformément répartis
- Repartitionnement coûteux

Phase	Période	Objectifs
Phase 1	Jan - Fév	OpenMP + tests de performances
Phase 2	Fév - Mars	MPI + décomposition domaine
Phase 3	Mars - Avril	Hybride + optimisations
Phase 4	Avril	Benchmarks + analyse
Rendu	~Avril 2025	Rapport

Outils et Environnement

- OpenMP 5.0, MPI 4.0
- Machines multi-coeurs locales
- Profiling : gprof, perf, Intel VTune

Métriques importantes :

1. **Speedup** : $S_p = \frac{T_1}{T_p}$ (temps séquentiel / temps parallèle)
2. **Efficacité** : $E_p = \frac{S_p}{p}$ (speedup / nombre de processeurs)
3. **Strong scaling** : Nombre de boid constant, augmenter le nombre de processeurs
4. **Weak scaling** : Charge par processeur constante, augmenter la taille du problème
5. **FPS maintenu** : Objectif 60 FPS avec 10000 boids

Benchmarks comparatifs : Séquentiel, OpenMP, MPI, Hybride

Bilan semestre 1

- Simulateur complet et fonctionnel (3 règles de Reynolds)
- Architecture modulaire et optimisée (SpatialGrid)
- Visualisation fluide (60 FPS @ 500 agents)

Objectifs Semestre 2

- **S2** : Passage à l'échelle via OpenMP/MPI
- Objectif : 10000+ boids temps réel
- Code source disponible : [lien vers GitHub](#)

- Reynolds, C. W. (1987). *Flocks, herds and schools : A distributed behavioral model*. SIGGRAPH '87.
- Reynolds, C. W. - Red3d Boids : <https://www.red3d.com/cwr/boids/>
- SFML Documentation : <https://www.sfml-dev.org/documentation/>
- OpenMP Documentation : <https://www.openmp.org/>
- MPI Standard : <https://www.mpi-forum.org/>
- GoogleTest Framework : <https://google.github.io/googletest/>

Merci de votre attention !

Questions ?